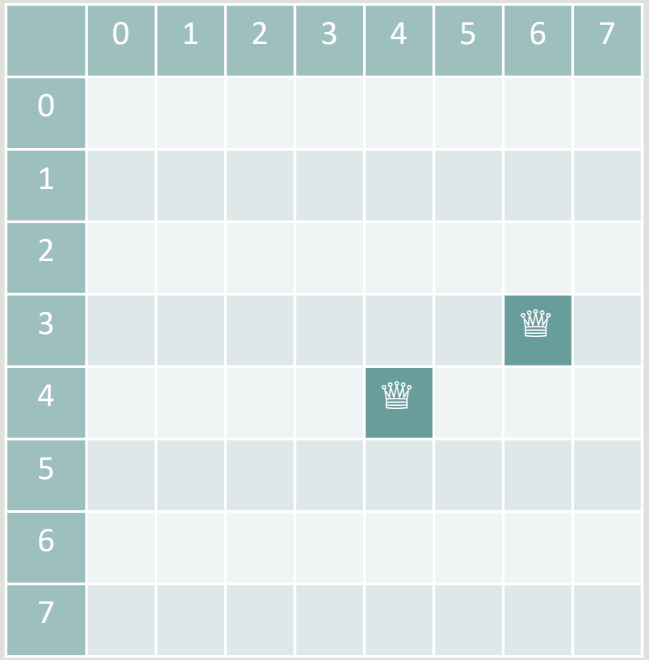
Constraint Satisfaction Problem Methods

for N-Queens

Early Implementation Ideas

Representing the Board

The board is a vector of integer pairs, where each pair represents the coordinates of a Queen.



$$\mathbf{B} = \{(\mathbf{4},\mathbf{4}),(\mathbf{6},\mathbf{3}),\dots\}$$

Complications

- The board must check the entire vector to know if a row/column is already taken.

→ What if the algorithm could access a row and know it has a Queen?

- A board is checked for validity in its current state, but it has no idea if a board will inevitably fail.

→ What if we maintained the possible placements of each Queen, so we know instantly when a Queen no longer has any valid position?

- Queens are always placed in the first available spot it sees.

→ What if we changed the order of placements to find a valid solution more efficiently?

Implementation with CSP methods

Representing the Board

Manage each row as its own variable
Each Queen $\boldsymbol{V_i}$ has domain $\boldsymbol{D_i}$ representing the columns it can be placed

	0	1	2	3	4	5	6	7
V_0								
V_1								
V_2								
V_3								
V_4								
V_5								
V_6								
V_7								

$D_0 = \{0, 1, 2, 3, 4, 5, 6, 7\}$
$D_1 = \{0, 1, 2, 3, 4, 5, 6, 7\}$
…

It is much easier to implement CSP methods with Queen Variables based on rows than coordinate pairs!

Backtracking

Stacks: Depth-First-Search (LIFO)

DFS adds Queens to the board until failure.
When a board fails, the stack "backtracks" to a board pushed earlier:

```
Stack csp;
push initial board to csp

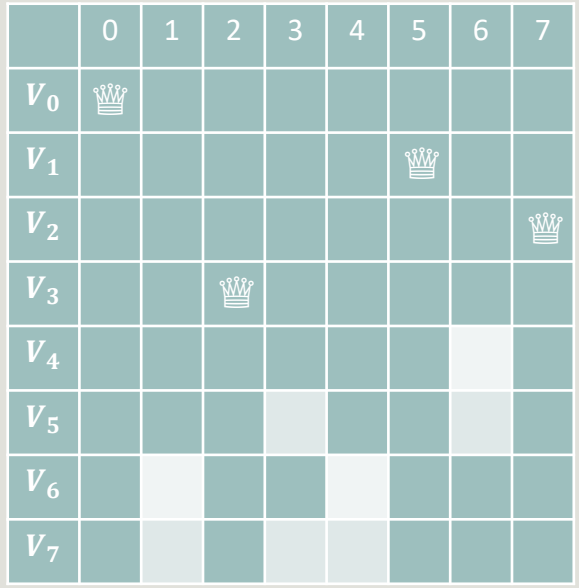
while (csp is not empty):
        if (csp is complete) return csp

        queen = next_queen()
        for (each value in queen's domain):
                if (no conflicts with csp):
                        add value to new board
                        push board to csp
```

CSP Methods: Filtering

Forward Checking

Update domains of other Queens.

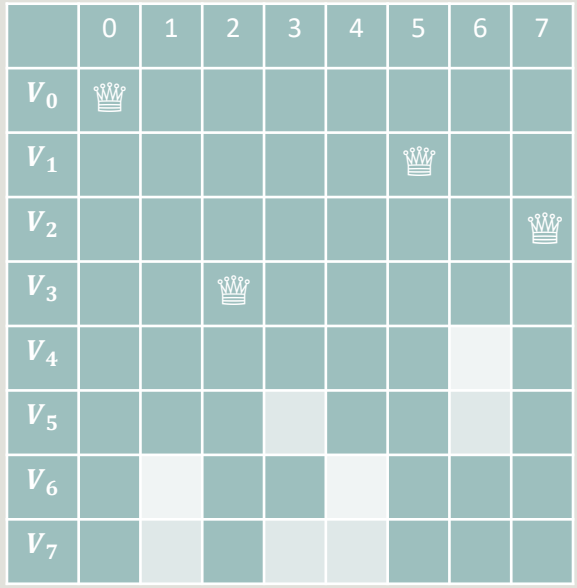


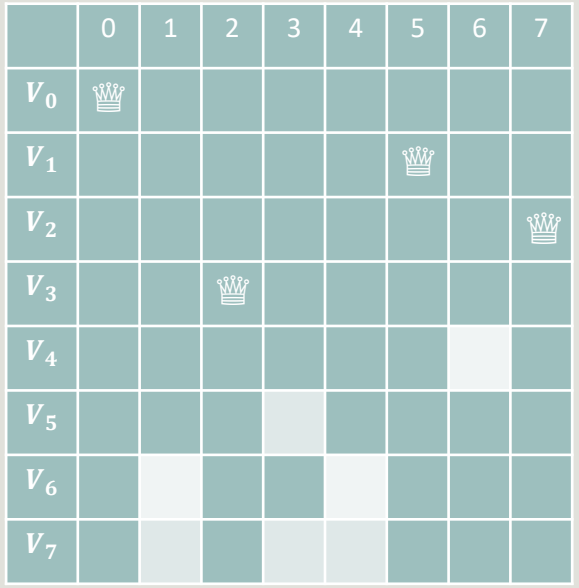
$D_0 = \{0\}$
$D_1 = \{2, 3, 4, 5, 6, 7\}$
…

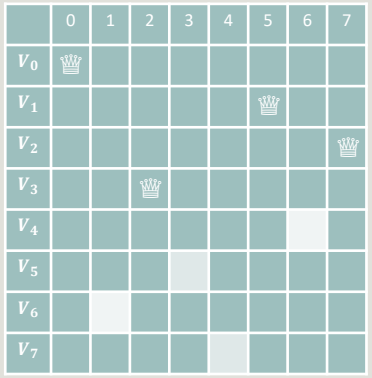
Arc Consistency (Constraint Propagation)

An arc $Q_i \rightarrow Q_j$ is consistent iff:

For all $i \in D_i$ there exists some $j \in D_j$ such that Q_i and Q_j don't conflict.







$Q_5 \rightarrow Q_4$:

If Q_5 were to pick column 6, Q_4's domain would become empty…

To make this arc consistent, we take column 4 out of Q_5's domain.

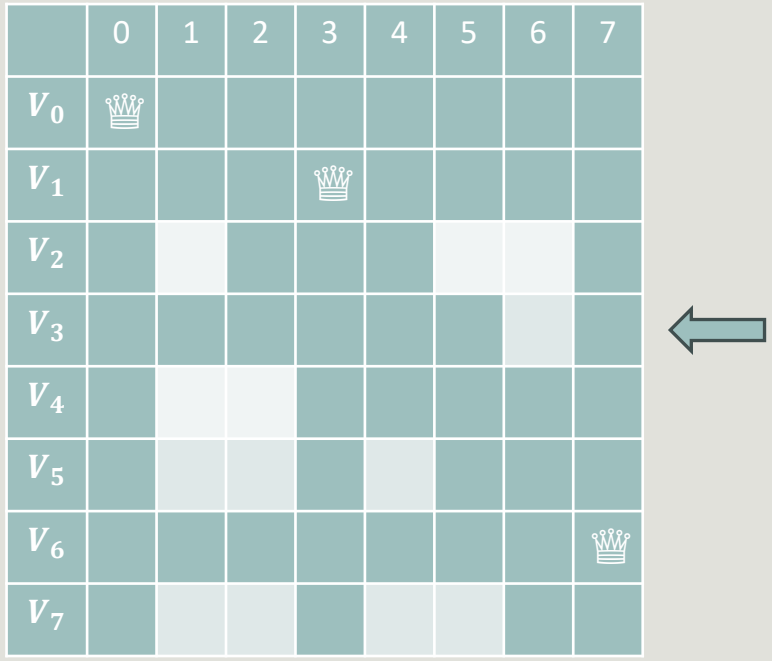
Also, Q_7 can't be in col 1 or 3, and Q_6 can't be in 4…

and now Q_7 can't be in col 1…

CSP Methods: Ordering

Minimum Remaining Values

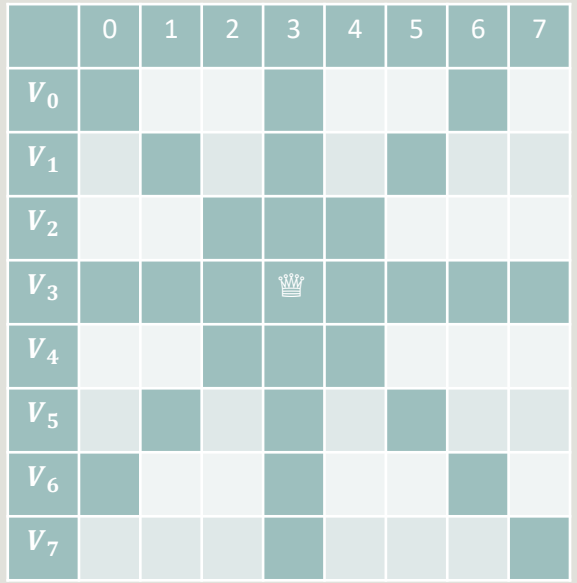
Pick the Queen with the smallest domain to place next



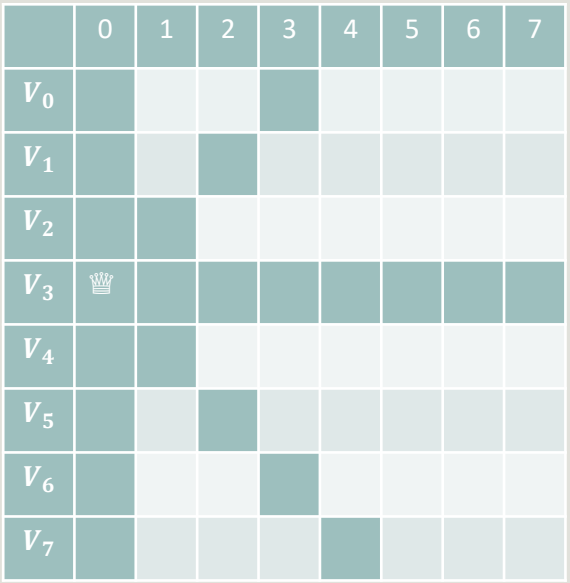
Note that the search problem won't update domains without some level of filtering.

Least Constraining Value

Pick the column placement that leaves the most possible placements for other Queens



Restricts 20 other placements…



… only 14 placements ♕

Performance: board size n = 25

Original

Execution time:
23,652.4 ms
Nodes expanded:
1,216,776 nodes

MRV

Execution time:
25,458.3 ms
Nodes expanded:
1,216,776 nodes

LCV

Execution time:
489,112 ms
Nodes expanded:
20,589,670 nodes

FC

Execution time:
833.378 ms
Nodes expanded:
19,097 nodes

ARC

Execution time:
857.581 ms
Nodes expanded:
2,547 nodes

Conclusion:

MRV & LCV suck...?

Without filtering, MRV behaves the exact same as picking Queens in sequential order. Similarly, LCV is **worse** because it wastes time deciding where to place a Queen.

Performance: board size n = 25

Original

Execution time:
23,652.4 ms
Nodes expanded:
1,216,776 nodes

FC

Execution time:
833.378 ms
Nodes expanded:
19,097 nodes

ARC

Execution time:
857.581 ms
Nodes expanded:
2,547 nodes

MRV

Execution time:
25,458.3 ms
Nodes expanded:
1,216,776 nodes

MRV + FC

Execution time:
9.402 ms
Nodes expanded:
214 nodes

MRV + ARC

Execution time:
17.48 ms
Nodes expanded:
59 nodes

LCV

Execution time:
489,112 ms
Nodes expanded:
20,589,670 nodes

LCV + FC

Execution time:
22.95 ms
Nodes expanded:
412 nodes

LCV + ARC

Execution time:
19.19 ms
Nodes expanded:
55 nodes

Performance: board size n = 25

Original

Execution time:
23,652.4 ms
Nodes expanded:
1,216,776 nodes

MRV

Execution time:
25,458.3 ms
Nodes expanded:
1,216,776 nodes

LCV

Execution time:
489,112 ms
Nodes expanded:
20,589,670 nodes

FC

Execution time:
833.378 ms
Nodes expanded:
19,097 nodes

MRV + LCV + FC

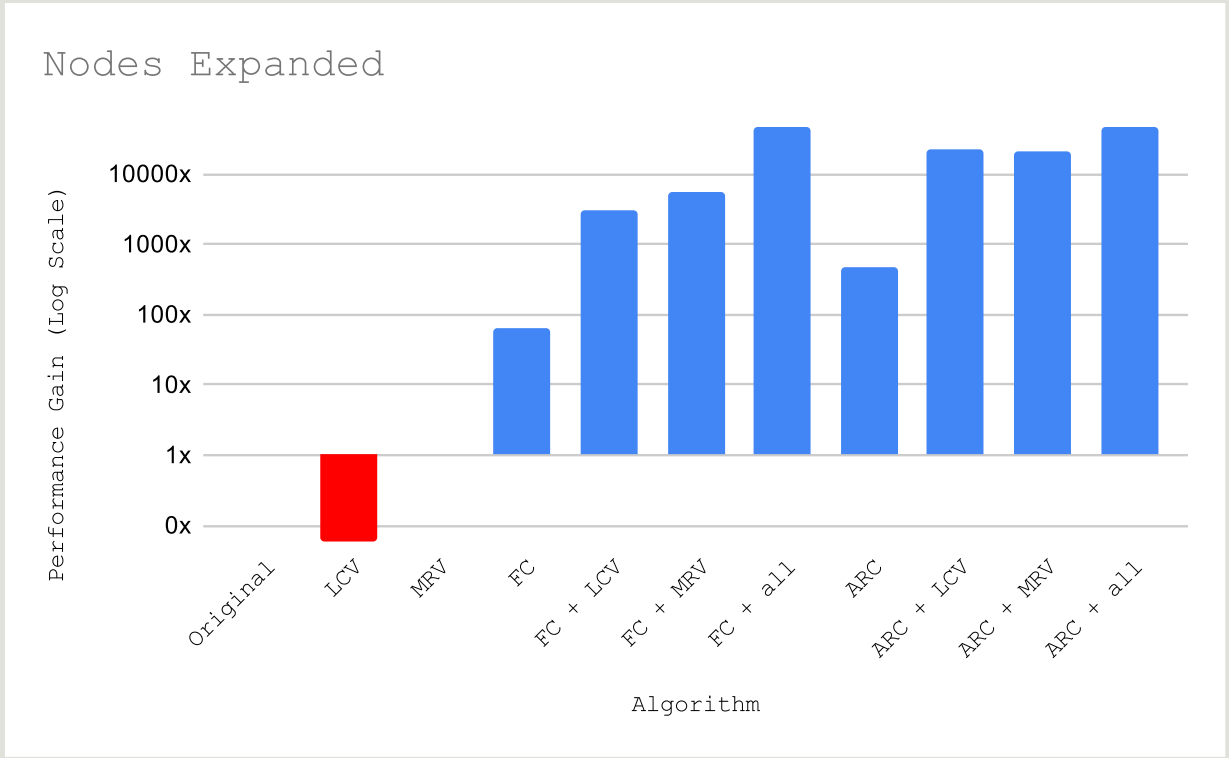
Execution time:
3.24645 ms
Nodes expanded:
26 nodes

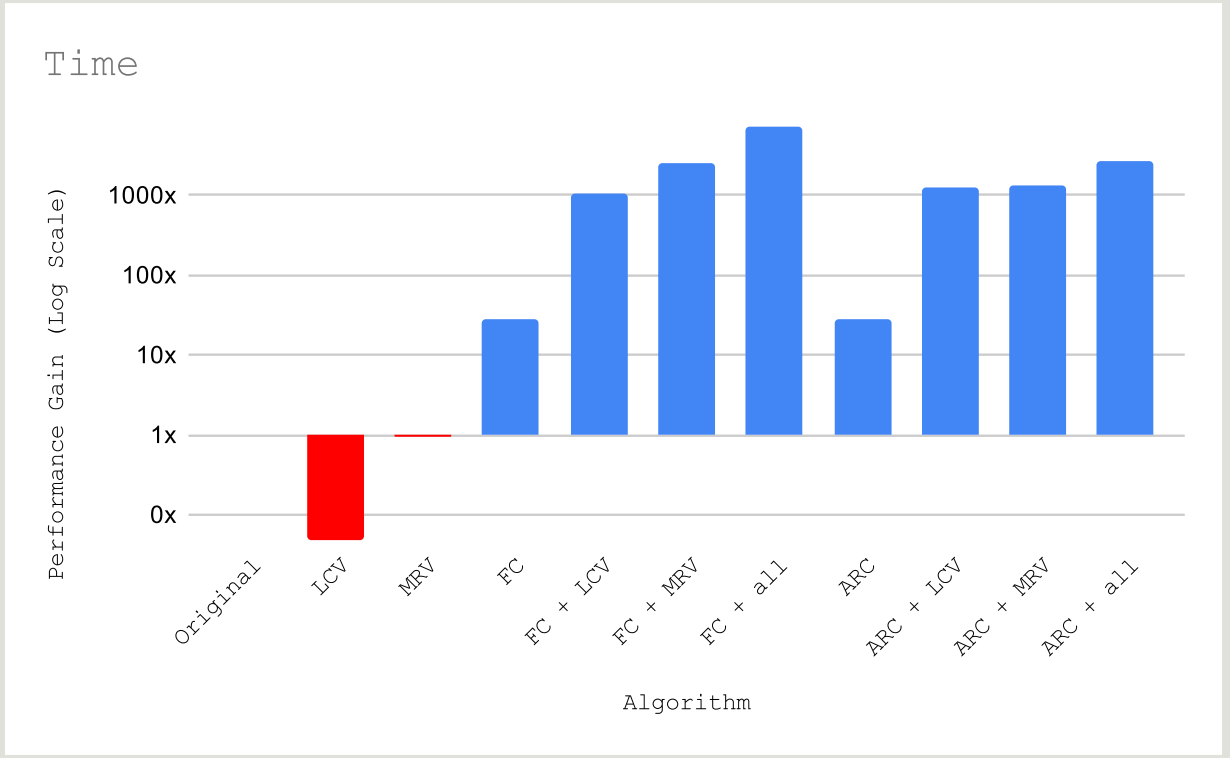
ARC

Execution time:
857.581 ms
Nodes expanded:
2,547 nodes

MRV + LCV + ARC

Execution time:
8.94307 ms
Nodes expanded:
26 nodes



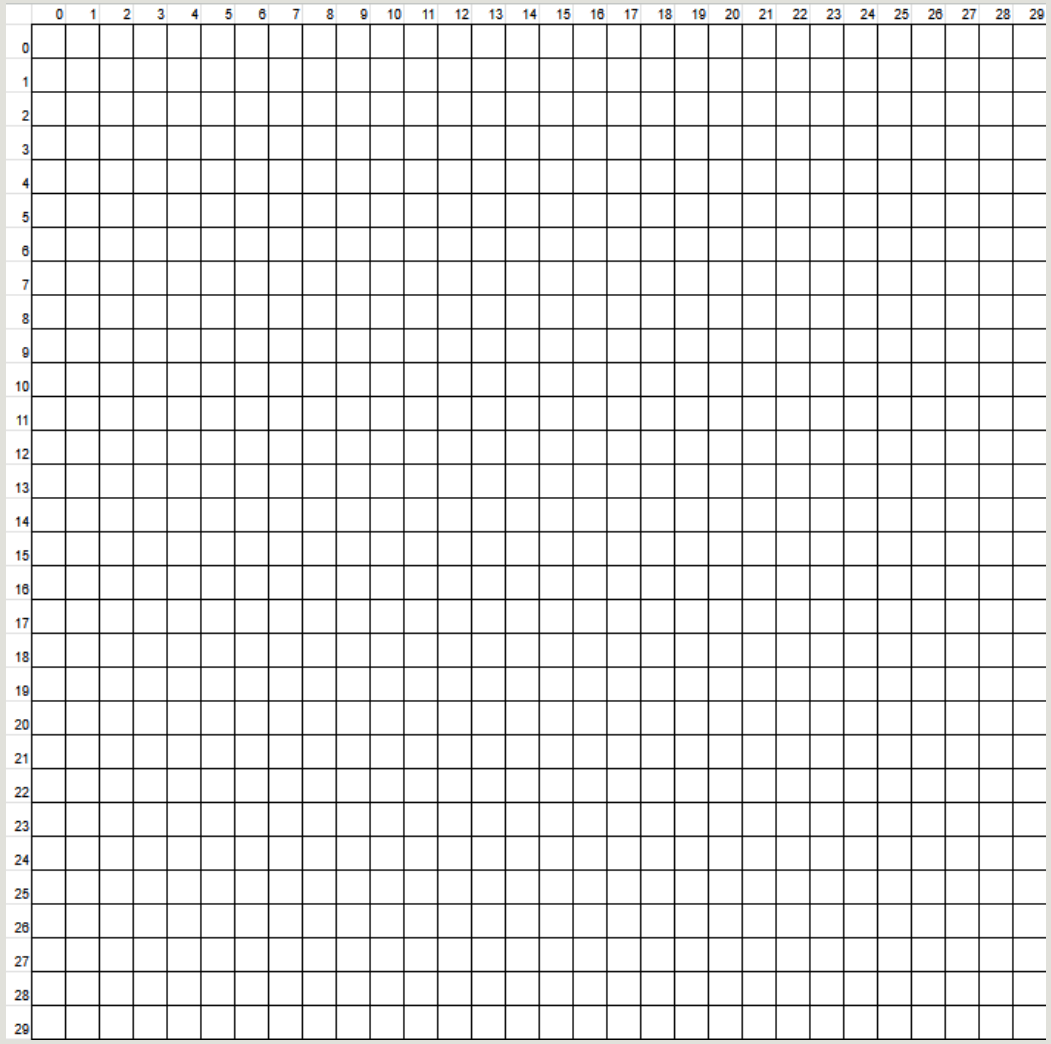


Performance: board size n = 25

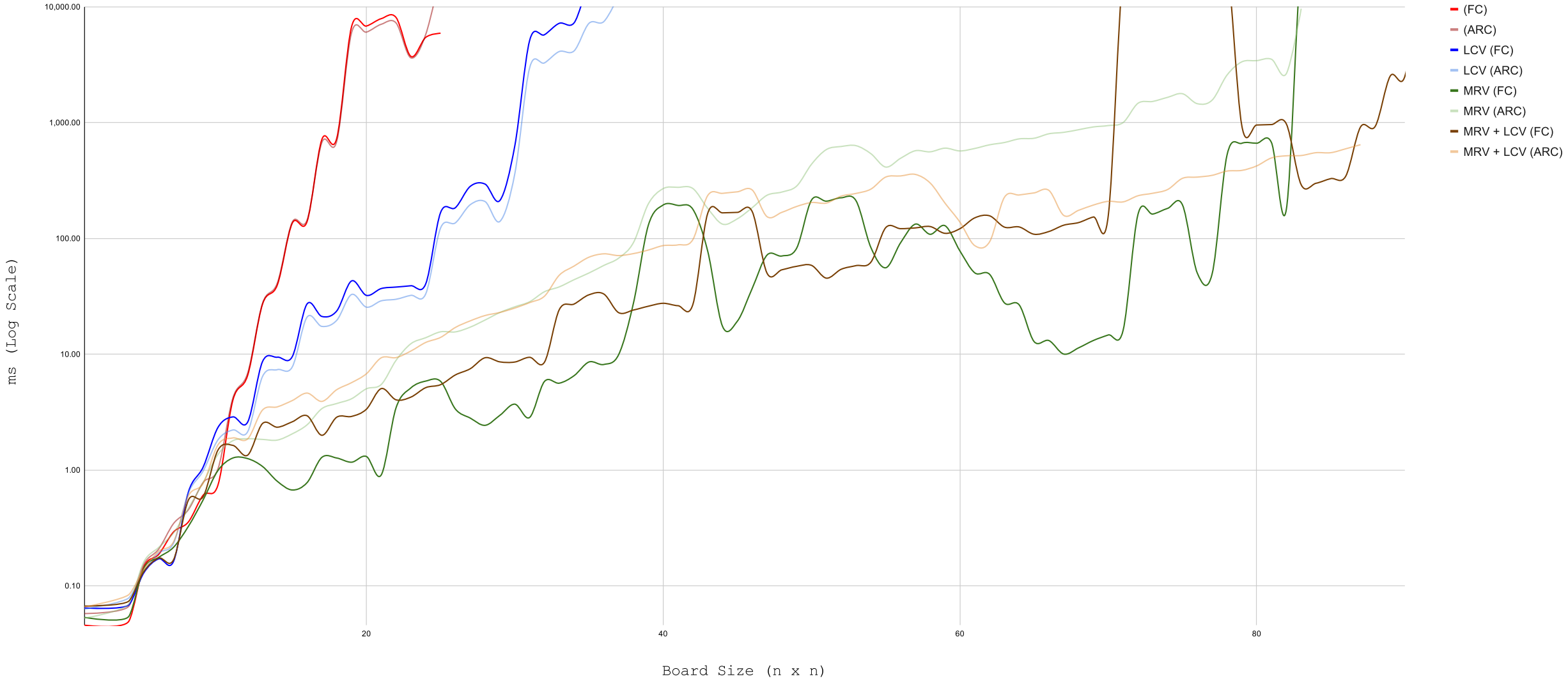
Performance: Demo size n = 30

ARC + MRV + LCV

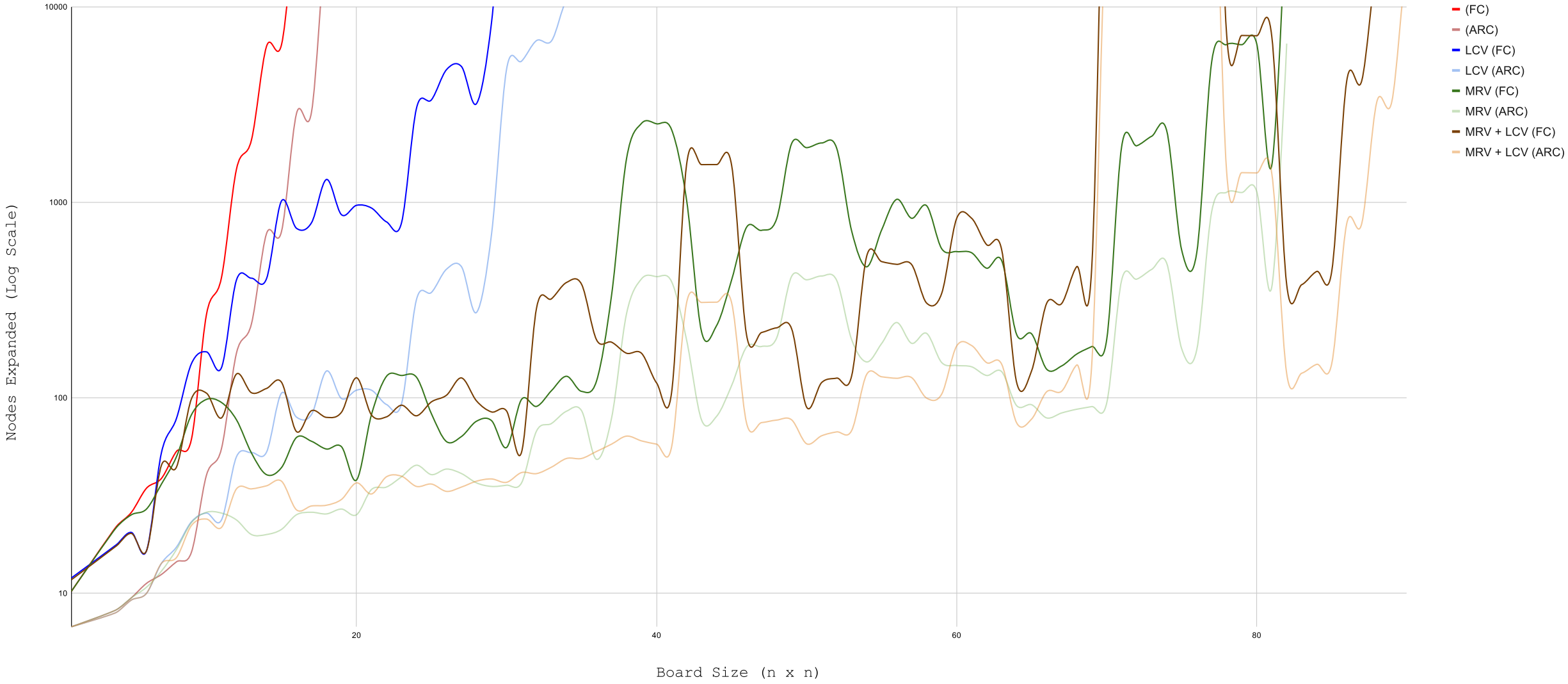
Execution time:
27.45 ms
Nodes expanded:
38 nodes



Performance: Time Taken Moving Average



Performance: Storage Moving Average



Observations

Time

FC is faster than ARC?

- ARC spends more time propagating through domains

LCV isn't always an improvement?

- On large boards, LCV can take longer to sort domains than FC+MRV alone takes to visit a lot of boards

MRV vs LCV?

- While LCV costs time to sort domains, our solver easily knows the size of each Queen's domain
- MRV is powerful because it "fails fast," and we only need a single solution.

Nodes Expanded

ARC visits less nodes than FC?

- ARC will eliminate more possible boards than FC without having to visit them
- A combination of ARC+MRV+LCV will visit a consistently low number of boards

However, it's still expensive to compute…

In general, as the board size increases, there will be a lot more variation in how long the algorithm takes to find a solution:
The solver is still able to get a solution on the first try, but the number of possible placements is still increasing quadratically - as well as the upper limit of how long it *might* take.

Memory or Time?

The Memory Bottleneck

While FC may be faster in a lot of situations, time is a theoretically infinite resource.
Memory, however, is not.

While you might be able to afford to wait 1,000,000 years, you may not even have the hardware to run an algorithm that demands too much space.

